GBDT vs MLP on the Bank Marketing Dataset

Advanced Machine Learning — Homework 2

1 Introduction

This report compares two fundamentally different modeling paradigms—Gradient Boosted Decision Trees (GBDT) and Multi-Layer Perceptrons (MLP)—on a binary classification task: predicting whether a client of a Portuguese bank will subscribe to a term deposit after a phone-based marketing campaign.

The **Bank Marketing** dataset is provided in two versions: an extended version with 20 input features, and a classic version with 17 features (16 inputs + 1 target). Since the dataset documentation only provides detailed descriptions for the 17-feature version, we opted to use the classic file (`bank-full.csv`) to ensure every feature is well-understood and interpretable. The selected dataset contains 45,211 records with 16 input features spanning client demographics (age, job, marital status, education), financial indicators (balance, credit default, housing/personal loans), and campaign attributes (contact type, month, number of contacts, outcome of previous campaigns). The target variable `y` is heavily imbalanced: only 11.7% of clients subscribed ("yes"), while 88.3% did not ("no").

This controlled experiment—same data, same splits, same evaluation—lets us directly compare how each model family learns, what preprocessing it demands, and where it excels or struggles.

2 Methods

2.1 Data Preparation

Missing values. No columns contained null values, but several categorical features (`job`, `education`, `contact`, `poutcome`) used the string `"unknown"` as a placeholder. We considered three strategies:

1. **Keep "unknown" as its own category** — simplest; may carry signal (clients who withhold information may behave differently).
2. **Impute with mode** — replaces unknowns with the most frequent value, but discards the "unknown" signal.
3. **Drop rows with unknowns** — cleanest data but loses ∼25% of rows.

We chose option 1 because "unknown" itself is informative: for instance, clients reached via an unknown contact method disproportionately came from older campaign records with lower subscription rates.

Feature engineering. Three derived features were added:

- `was_contacted_before`: binary flag from `pdays` (1 if previously contacted, 0 otherwise). 82% of clients had no prior contact.
- `age_group`: age binned into five groups (young, mid, senior, elder, retired).
- `balance_positive`: binary flag for positive account balance.

Duration removal. The `duration` feature (last contact duration in seconds) was dropped. While the dataset authors note it is highly predictive, it is only available *after* a call ends and therefore cannot be used in a realistic deployment scenario. Removing it forces the models to learn from features that are available before the campaign call is made.

Encoding. We considered three encoding strategies: (a) one-hot encoding, (b) label encoding, and (c) a mix (label for binary columns, one-hot for multi-class). We chose **one-hot encoding with `drop_first=True`** for all categoricals. This avoids implying ordinal relationships (e.g., "admin" < "student") that could mislead gradient-based MLP training, while the dropped reference category prevents multicollinearity. The result was 47 input features.

Train / Validation / Test split. A stratified 70 / 15 / 15 split yielded 31,647 training, 6,782 validation, and 6,782 test samples. Stratification preserved the ∼11.7% positive rate across all three sets.

Feature scaling. A `StandardScaler` was **fit on the training set only** and then applied to validation and test sets, preventing data leakage.

Scaling is essential for MLP because its neurons compute weighted sums of inputs followed by non-linear activations; features on vastly different scales (e.g., `balance` ranging to 102,127 vs. binary indicators) elongate the loss surface, causing slow, unstable gradient descent. Standardization (mean= 0, std= 1) makes the loss landscape more isotropic.

Decision trees, by contrast, split on individual features using thresholds ("is balance > 500?"). These splits are invariant to monotonic transformations—whether balance ranges from 0–100,000 or is standardized to −2 to +3, the tree finds the same optimal split. Scaling is therefore unnecessary for GBDT.

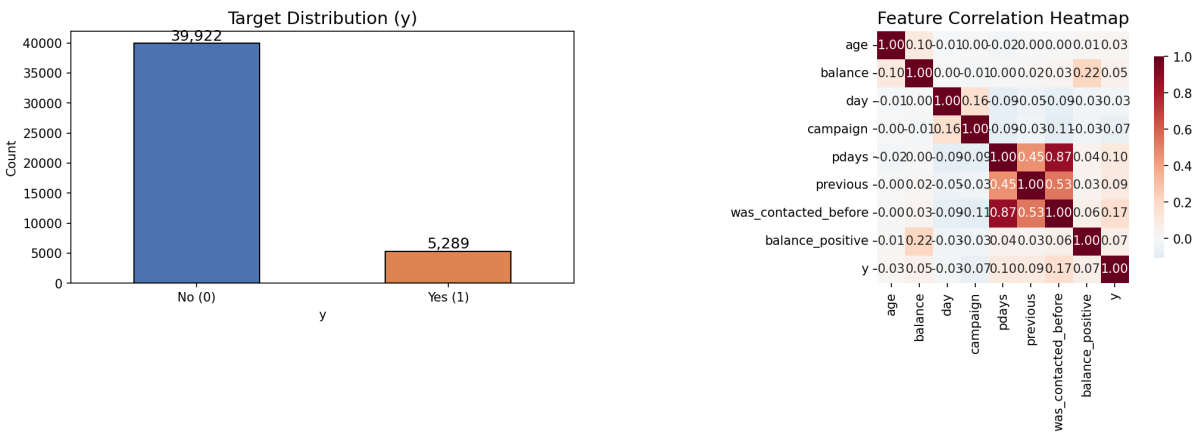

Figure 1: Left: class distribution showing 88:12 imbalance. Right: Pearson correlation heatmap of numeric features with the target. Most individual correlations are weak ($|r| < 0.15$), motivating non-linear models.

2.2 GBDT Implementation (XGBoost)

We used `XGBClassifier` from the `xgboost` library. To address class imbalance, we set `scale_pos_weight` $= N_{\text{neg}}/N_{\text{pos}} \approx 7.5$, which up-weights the minority class in the loss function—analogous to cost-sensitive learning.

Baseline. The initial model used `learning_rate=0.1`, `n_estimators=500`, `max_depth=5`, `subsample=0.8`. It achieved validation F1 = 0.45 for the positive class and trained in under 1 second. The training-

vs-validation loss curve (Figure 2) showed the model was still improving at round 490, with a train–val loss gap of ∼0.09—indicating mild overfitting at most.

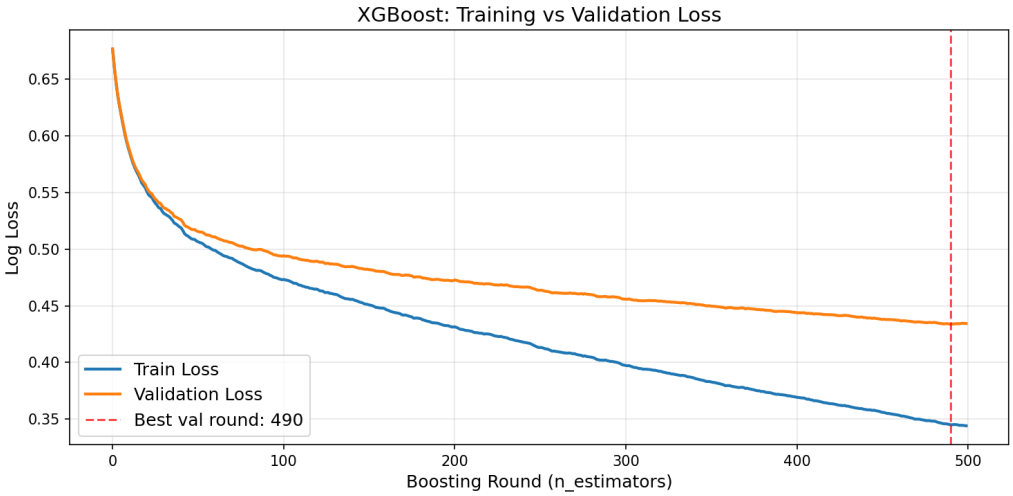

Figure 2: XGBoost baseline: training vs. validation log-loss over 500 boosting rounds. The red dashed line marks the round with minimum validation loss.

Learning rate study. We compared three learning rates (0.01, 0.1, 0.3) with 500 trees each (Figure 3):

Learning Rate	Val Accuracy	Val F1	Best Round
0.01	0.806	0.437	499 (still improving)
0.10	0.830	0.450	490
0.30	0.842	0.418	490

Table 1: Effect of learning rate on XGBoost validation performance.

A low rate (0.01) converges slowly and needs more trees; a high rate (0.3) reaches lower loss but sacrifices F1—the model becomes more conservative on positive predictions, favoring raw accuracy over recall. The rate 0.1 offered the best F1 trade-off.

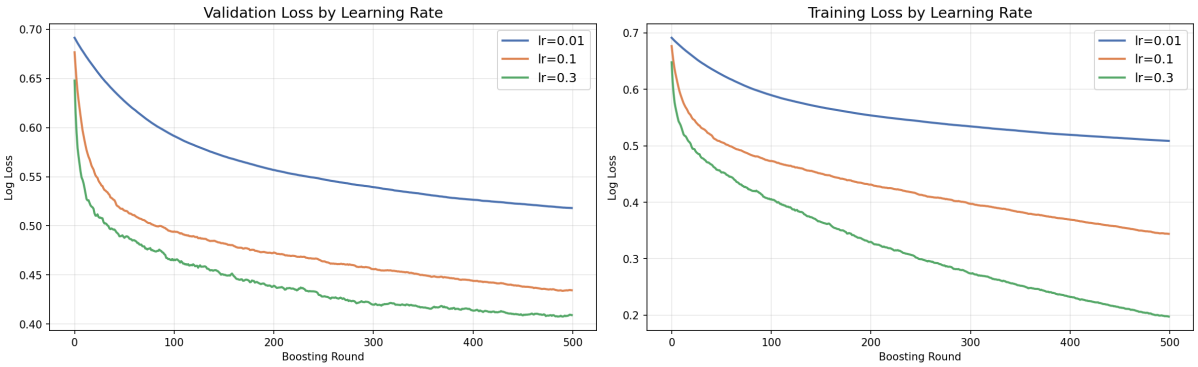

Figure 3: Validation loss (left) and training loss (right) for three learning rates. Lower rates converge more slowly but generalize comparably; higher rates risk overfitting earlier.

Hyperparameter tuning. `RandomizedSearchCV` explored 30 combinations over `learning_rate`, `n_estimators`, `max_depth`, `subsample`, `reg_alpha`, and `reg_lambda`, using 3-fold CV with F1

scoring. The best configuration was: `lr=0.05, n_estimators=200, max_depth=7, subsample=0.7, reg_lambda=2.0` (CV F1 = 0.446).

The tuned model achieved validation precision = 0.37, recall = 0.63, F1 = 0.47—a modest improvement over the baseline, confirming that GBDT is robust to hyperparameter choices on this dataset.

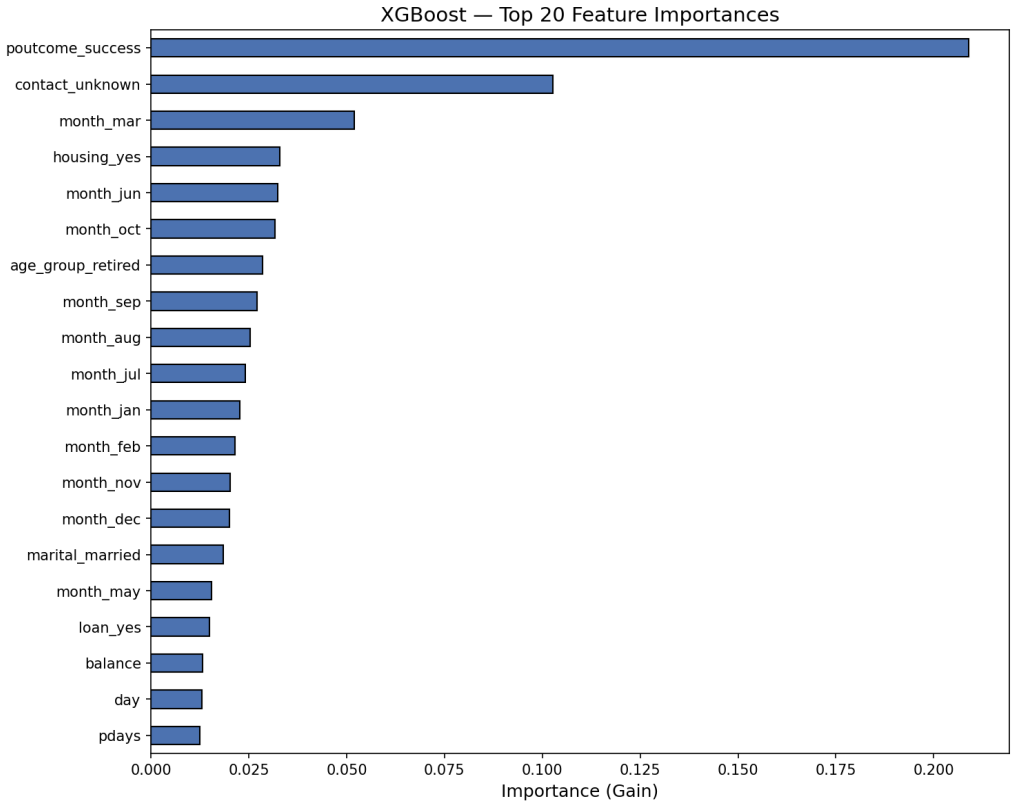

Figure 4: Top 20 feature importances from the tuned XGBoost model. Previous campaign outcome (`poutcome`), contact method, campaign month, and account balance dominate.

2.3 MLP Implementation

We used `MLPClassifier` from scikit-learn with standardized features.

The class imbalance problem. Our first MLP (128 neurons, relu, lr = 0.001) converged in only **34 iterations** with a validation accuracy of 0.90—but recall for "Yes" was just 0.24 and F1 was 0.34. The validation score plateaued at ∼0.9 from the first iteration because the model immediately learned to predict "No" for nearly everything, achieving 88% accuracy trivially.

Unlike XGBoost, scikit-learn's `MLPClassifier` has no `class_weight` parameter. We addressed this by **oversampling the minority class**: we randomly resampled the positive class (with replacement) to match the majority, creating a balanced training set of 55,890 samples (50/50 split). After oversampling, training extended to 235 iterations with meaningful loss descent, and recall for "Yes" improved from 0.24 to 0.48.

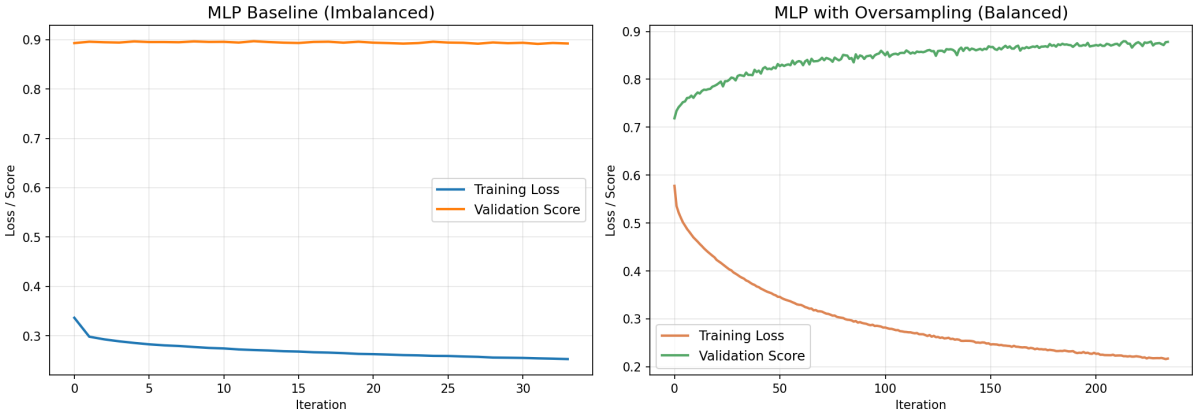

Figure 5: MLP loss curves before (left) and after (right) minority oversampling. Without balancing, the validation score flatlines immediately. With balancing, training shows genuine convergence.

Architecture comparison. We tested four architectures on the balanced data:

Architecture	Val Accuracy	Val F1	Iterations
(64,)	0.774	0.360	142
(128,)	0.797	0.357	235
(128, 64)	0.833	0.359	176
(256, 128, 64)	0.851	0.339	147

Table 2: Effect of network depth/width. Accuracy increases with depth but F1 *decreases*—deeper networks use extra capacity to predict "No" more confidently rather than improving minority-class detection.

Figure 6: Training loss curves for different MLP architectures. Deeper networks achieve lower training loss but do not generalize better on the minority class.

Configuration	Val Accuracy	Val F1	Iterations
relu, lr=0.001	0.797	0.357	235
relu, lr=0.01	0.771	0.331	87
relu, lr=0.1	0.799	**0.412**	49
tanh, lr=0.001	0.815	0.331	300
tanh, lr=0.01	0.816	0.315	113

Table 3: Learning rate and activation comparison. The aggressive relu lr=0.1 achieved the best F1 (0.41) by escaping the local minimum where the model defaults to predicting "No."

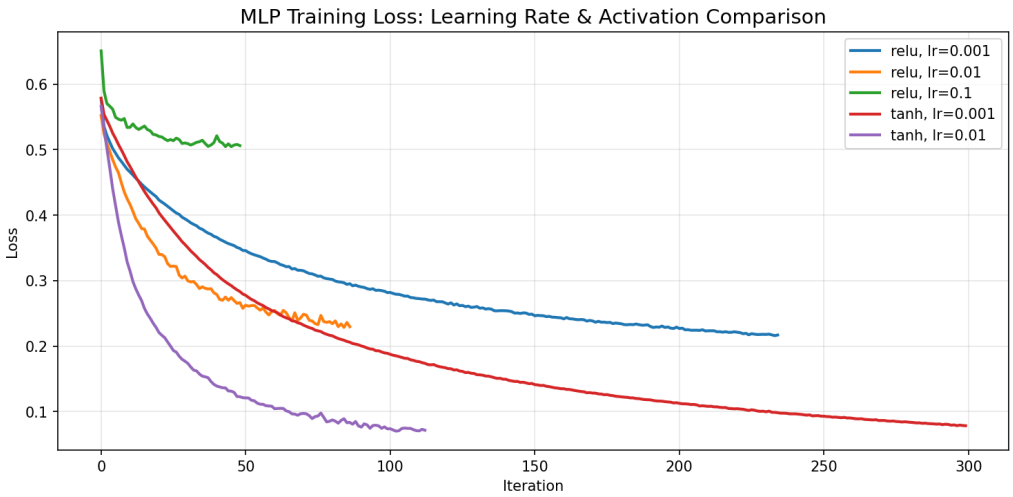

Figure 7: Training loss curves for different learning rates and activation functions. Higher learning rates converge faster but with more noise; tanh converges more smoothly but achieves worse F1.

Learning rate and activation study.

Hyperparameter tuning. `RandomizedSearchCV` over architectures, activations, learning rates, and L2 regularization strength yielded: `(128, 64)`, `tanh`, `lr=0.001`, `alpha=0.001`. The CV F1 of 0.93 was measured on the balanced (oversampled) data and is therefore inflated; the real validation F1 on original data was 0.33.

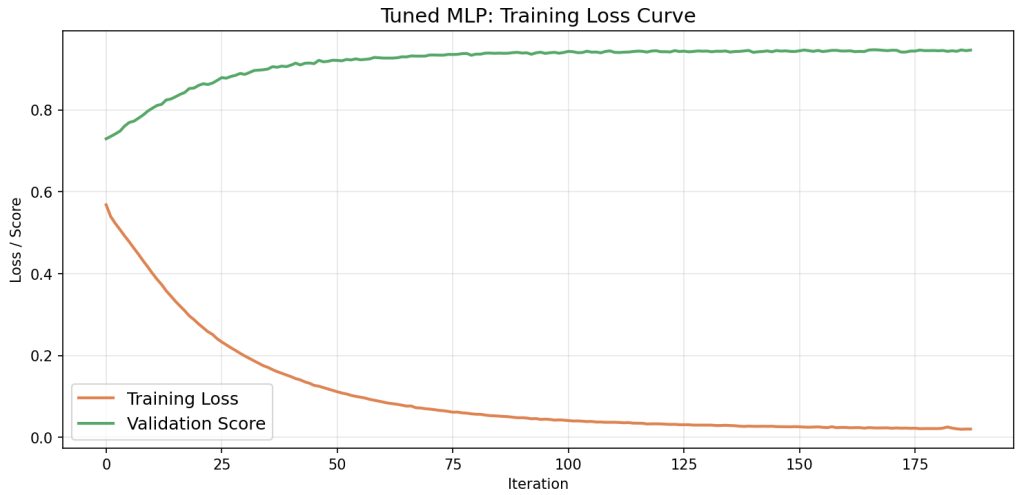

Figure 8: Training loss and validation score for the final tuned MLP.

3 GBDT vs MLP Comparison

Both final models were evaluated on the held-out test set (6,782 samples), used for the first time at this stage.

Metric	GBDT (XGBoost)	MLP	Winner
Accuracy	0.823	0.833	MLP
Precision	0.350	0.301	GBDT
Recall	0.600	0.325	GBDT
F1-score	0.442	0.313	GBDT
AUC-PR	0.461	0.264	GBDT
Training Time	0.32 s	32.28 s	GBDT

Table 4: Test set comparison. GBDT wins on every metric except raw accuracy, which is misleading under class imbalance (predicting all "No" gives 88% accuracy). The AUC-PR gap (0.46 vs. 0.26) shows GBDT maintains better precision–recall trade-offs across all classification thresholds. GBDT is also 100× faster to train.

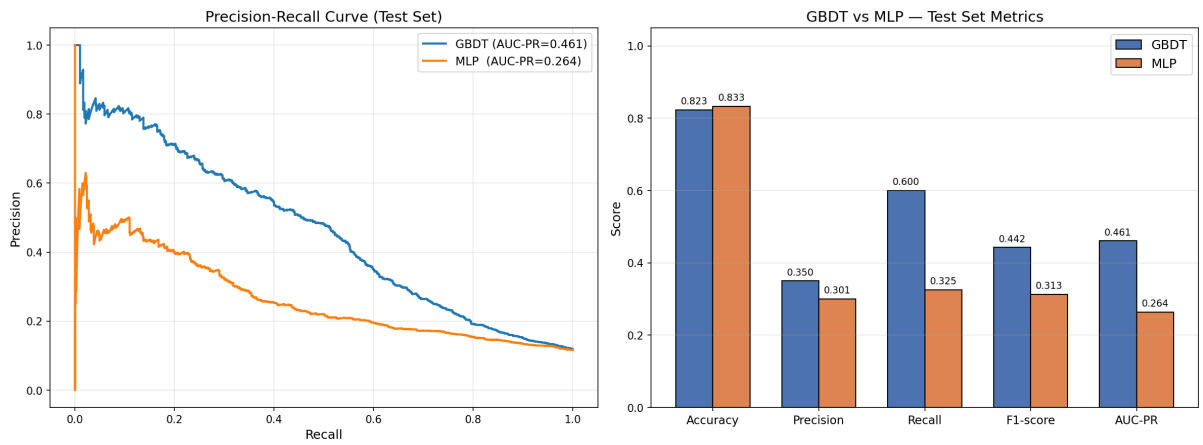

Figure 9: Left: Precision–Recall curves on the test set. GBDT dominates across all recall levels. Right: side-by-side bar chart of key metrics.

3.1 When to prefer GBDT vs. MLP

GBDT excels on **structured, tabular data** with mixed feature types, class imbalance, and interpretability requirements. It requires minimal preprocessing, trains extremely fast, and has built-in regularization. MLP may be preferred for very large datasets with complex non-linear interactions among continuous features, or for inherently high-dimensional inputs (embeddings, images, time-series). For this medium-sized tabular dataset with categoricals and imbalance, GBDT is the clear winner.

3.2 Interpretability

GBDT provides **built-in feature importance** (Figure 4). We identified previous campaign outcome, contact method, campaign month, and account balance as top predictors—directly actionable for the bank's marketing strategy. MLP is a black box: the learned weights across multiple hidden layers do not map to intuitive, feature-level explanations. Post-hoc techniques (SHAP, permutation importance) can approximate interpretability but add complexity and are not inherent to the model.

3.3 Handling categoricals and missing values

XGBoost handles missing values **natively** by learning optimal split directions for absent data. It also works well with encoded categoricals since tree splits impose no ordinal assumptions. MLP requires all features to be numeric and standardized—categoricals must be one-hot encoded and missing values imputed *before* training, making the pipeline more complex and introducing additional design choices.

3.4 Sensitivity to hyperparameters

MLP proved **significantly more sensitive**. Our experiments showed:

- F1 ranged from 0.31 to 0.41 across learning rates and activations.
- Without class balancing, recall for "Yes" was only 0.24—the model simply did not learn the minority class.
- Deeper architectures *hurt* F1 despite lowering training loss.

GBDT was robust: the baseline with near-default parameters already achieved $F1 = 0.45$, and tuning provided only marginal improvement. GBDT's built-in regularization (`max_depth`, `subsample`, `reg_lambda`) makes it naturally resistant to overfitting without careful manual tuning.

4 Discussion

4.1 Bias–Variance Reflection

GBDT reduces bias through **sequential boosting**—each new tree corrects the residual errors of the ensemble so far—and controls variance through regularization (tree depth limits, row/column subsampling, L1/L2 penalties). This combination is well-suited to medium-sized tabular data.

MLP reduces bias through **flexible function approximation** (universal approximation theorem) but is prone to high variance, especially with limited data and class imbalance. The oversampling strategy addressed the imbalance but introduced its own variance: duplicated minority samples can cause the network to memorize specific instances rather than learning generalizable patterns. This was reflected in the inflated CV F1 (0.93 on balanced data) versus the much lower test F1 (0.31 on real data).

4.2 Key lessons learned

1. **Class imbalance must be handled explicitly.** For GBDT, `scale_pos_weight` provided a clean, built-in solution. For MLP, the lack of a `class_weight` parameter forced us to oversample, which was effective but added pipeline complexity and inflated cross-validation scores.
2. **Accuracy is misleading under imbalance.** MLP achieved higher accuracy (83.3%) than GBDT (82.3%), but this was entirely due to more aggressive "No" predictions. F1, precision–recall, and AUC-PR are far more informative.
3. **Simpler architectures can outperform deeper ones.** For MLP, adding layers increased accuracy but *decreased* F1—the extra capacity was used to predict the majority class more confidently.
4. **Feature scaling is non-negotiable for neural networks.** Without standardization, MLP convergence is slow and unstable. Trees are invariant to feature scale.

4.3 Limitations

- **Duration removed:** including it would dramatically improve both models but at the cost of realism.
- **Oversampling artifacts:** random oversampling with replacement can cause overfitting. SMOTE or class-weighted loss functions (e.g., via PyTorch) could be explored as alternatives.
- **MLP framework:** scikit-learn's `MLPClassifier` lacks modern regularization (dropout, batch normalization, learning rate scheduling). A PyTorch implementation could close the performance gap with GBDT.
- **Feature engineering:** only basic derived features were created. Interaction terms and campaign timing patterns could improve both models.

5 AI Tool Disclosure

- **Tool used:** Claude (Anthropic), via Claude Code CLI.

What it contributed:

- Generating the majority of the code for data loading, cleaning, model training, hyperparameter tuning, and evaluation.
- Suggesting different preprocessing options (e.g., how to handle "unknown" values, whether to keep or drop the `duration` column, which encoding strategy to use for categoricals).
- Implementing class imbalance handling (oversampling for MLP, `scale_pos_weight` for XGBoost) and explaining trade-offs.
- Drafting the LaTeX report structure and text based on my guidance.

My own contributions:

- Preparing the workspace: downloading and extracting the dataset, creating the Python notebook, and selecting the 17-feature dataset version.
- Making all modeling decisions: choosing to keep "unknown" as its own category, dropping `duration` for realism, selecting one-hot encoding over label encoding, and deciding on feature engineering choices.
- Reviewing, running, and verifying all code and outputs cell by cell.
- Reviewing and understanding training and evaluation results—such as identifying the MLP validation score plateau, noticing the deeper-networks-hurt-F1 pattern, and questioning the flat validation curve.

- Instructing Claude on what to discuss in each section of the report and requesting specific adjustments to content and formatting.